

SCHEDULING PARALLEL PROCESSORS TO MINIMIZE SETUP TIME

ROBERT T. SUMICHRAST*

Department of Management Science, Virginia Polytechnic Institute and State University, Blacksburg, VA 24061, U.S.A.

(Received September 1985; revised October 1986)

Scope and Purpose—A new algorithm for scheduling is proposed. This method is appropriate for make-to-stock industries with high and sequence-dependent setup costs, identical parallel machines, and a product mix that varies from one planning horizon to the next.

Abstract—The problem of scheduling parallel processors in a make-to-stock environment with sequence-dependent setup costs is considered. A new algorithm which formulates a series of 0/1 integer subproblems is proposed and contrasted with an earlier formulation. Parallels between the subproblem formulations and generalized networks are discussed.

INTRODUCTION

Scheduling machines has long been a subject of interest for production management. One situation which has frequently been studied is the scheduling of identical parallel machines. Several studies have been done when the objective is to meet demand while minimizing a tardiness function through algorithmic [1-3] and heuristic [4] procedures. However, the literature has paid less attention to scheduling problems in industries with high setup costs and timing flexibility. These conditions have been observed in weaving operations for both fiberglass and textile manufacturers, and for fiberglass mat production.

Previously published research in this area concentrated on finding heuristic solutions. Oliff [5] developed a sequencing heuristic for the production of a fiberglass mat product. This heuristic classified products into groups and sequenced within groups by decreasing width. It was developed as part of a larger hierarchical scheduling system. A major producer of this product implemented this system and reports substantial cost savings have resulted.

Dearing and Henderson [6] modeled loom assignment in a textile weaving operation as an integer linear program (ILP) problem, but solved the problem using linear programming and rounding. Their formulation considered the cost of setup changes only as a constraint while trying to minimize deviations from production goals.

Burch and Sumichrast [7] considered scheduling looms; each of which is capable of weaving several different widths of fiberglass cloth simultaneously. One heuristic was developed to assign the type of product to weave and the run length to individual looms and a second to load needed widths onto the looms efficiently.

This paper begins with a description of the problem to be solved: including explicit consideration of simplifying assumptions and a mathematical model (adapted from Dearing and Henderson [6]). The difficulties associated with solving this model directly through linear programming are noted and serve as motivation for a new solution methodology. Therefore, a specialized solution method is presented and illustrated through an example. A discussion of computational effort is used to suggest the type of problem best solved using this methodology. Finally, the relaxation of key assumptions is considered and directions for future research suggested.

*Robert T. Sumichrast is an Assistant Professor of Management Science at Virginia Polytechnic Institute, Blacksburg. He received the Ph.D. in Management Science from Clemson University and a B.S. in Physics from Purdue University. He is certified in Production and Inventory Management by the American Production and Inventory Control Society. Dr Sumichrast's primary research and teaching interests are in the areas of production and operations management. He has published in *Computers & Operations Research*, *Production and Inventory Management*, *The Applied Journal of Management Science* and *Industrial Management*. He has served as a consultant for Owens-Corning Fiberglas Corporation.

PROBLEM DESCRIPTION

The problem is to determine how to schedule production on m identical machines so that demand for each of n products is met and setup cost is minimized. Both demand and production capacity are measured in machine-days and it is assumed that both are known. Machines are always setup to produce one of the products and the initial setups are known. During the fixed planning horizon, total demand must equal production capacity and this capacity is not diminished due to nonproductive time as machines are switched between products. Setup changes occur only at the start of a day and there is no limit to the number of changes which can be made during one day. Finally, the cost to switch between two products is known, independent of the direction of the change (i.e. the cost to switch from product i to j is the same as the cost to switch from j to i for all i and j), and the costs satisfy the triangle inequality; it is always cheaper to switch directly from product i to j , rather than switching from i to j through some intermediate product, k .

This situation can be modeled as an integer linear programming (ILP) formulation. The ILP has the structure of a network with side constraints which force production. The model is stated below.

$$\text{minimize} \quad \sum_{i=1}^n \sum_{j=1}^n C_{ij} \sum_{d=1}^H Z_{ijd}$$

subject to

$$X_{ik} + \sum_{j=1}^n Z_{ikj} - \sum_{i=1}^n Z_{lik} = S_k \quad \text{for } k = 1, 2, \dots, n, \quad (1)$$

$$X_{dk} + \sum_{j=1}^n Z_{dkj} - \sum_{i=1}^n Z_{dik} = X_{d-1,k} \quad \text{for } k = 1, 2, \dots, n \text{ and } d = 2, 3, \dots, H \quad (2)$$

$$\sum_{d=1}^H X_{dk} = D_k \quad \text{for } k = 1, 2, \dots, n, \quad (3)$$

where:

X_{di} = the number of machines producing product i on day d ,

Z_{dij} = the number of machines switched from i to j on day d ,

S_i = the number of machines initially setup to produce product i ,

c_{ij} = the cost to switch a machine from product i to j ,

D_i = the demand for product i (in machine-days),

H = the number of days in the planning horizon,

n = the number of products to be scheduled, and all variables are non-negative integers.

Most of the network structure of this formulation makes it possible to display the situation (except for the production constraints) graphically. Figure 1 illustrates this; a three-product 2-day horizon problem has been modeled. One can interpret this model as a flow of machines through product-days, where equations (1) and (2) represent conservation of flow on the network. The remaining set of constraints forces production in the desired amounts.

It is impractical to obtain an optimal integer solution to this formulation unless the number of products and machines is small. Rounding an LP solution may be considered only if a large number of changes is needed and only if a limit on the number of changes per day is added to prevent all changes from being scheduled on the first day. However, for plants which have relatively few changes during each planning period, rounding to an integer solution is totally unacceptable. The next section provides a method for obtaining a good integer solution to the above formulation.

SOLUTION METHOD

Rather than solving the full problem of scheduling all production, this procedure considers only resources that must be shifted. A series of small 0/1 integer problems will be solved in place of one

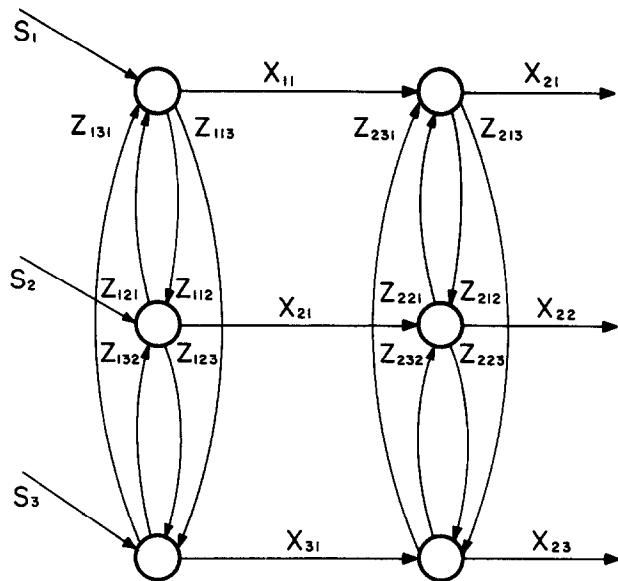


Fig. 1

large ILP. The intuitive benefits of a graphical description as shown in Fig. 1, are still largely maintained, as a series of networks with gains result from the new algorithm. Steps of the algorithm are summarized in the flowchart of Fig. 2 and are explained below.

Determine capacity needs

First, find the excess capacity or capacity deficiency associated with each of the n products. This is a simple matter since the demand for each product—measured in machine-days—is known and the capacity for production without any setup changes is the product of the planning horizon, and the number of machines initially setup for the product. The amount of excess production associated with a product is then used to determine the greatest number of machines that can leave the product in an optimal solution.

It is clear that in an optimal solution, machines will seldom be transferred from a product i , and later in the horizon, be transferred back to product i . The maximum number of machines that should be assigned to any product i , at any time is the smallest number that can product D_i during the horizon and is given by M_i where:

$$M_i = \lceil D_i/H \rceil.$$

The brackets indicate the smallest integer greater than or equal to the quantity enclosed. Therefore, the maximum number that should leave product i is L_i , where:

$$L_i = \lceil S_i - D_i/H \rceil.$$

Setup ILP subproblem

The main step is the formulation and solution of a set of 0/1 integer linear programming problems. Each subproblem minimizes the cost of meeting the deficiency of a single product k , from products with surplus capacity and is stated below.

$$\text{minimize } \sum_i \sum_r C_{ik} Z_{irk}$$

subject to

$$\sum_i \sum_r E_{ir} Z_{irk} - \delta_k = \Delta_k,$$

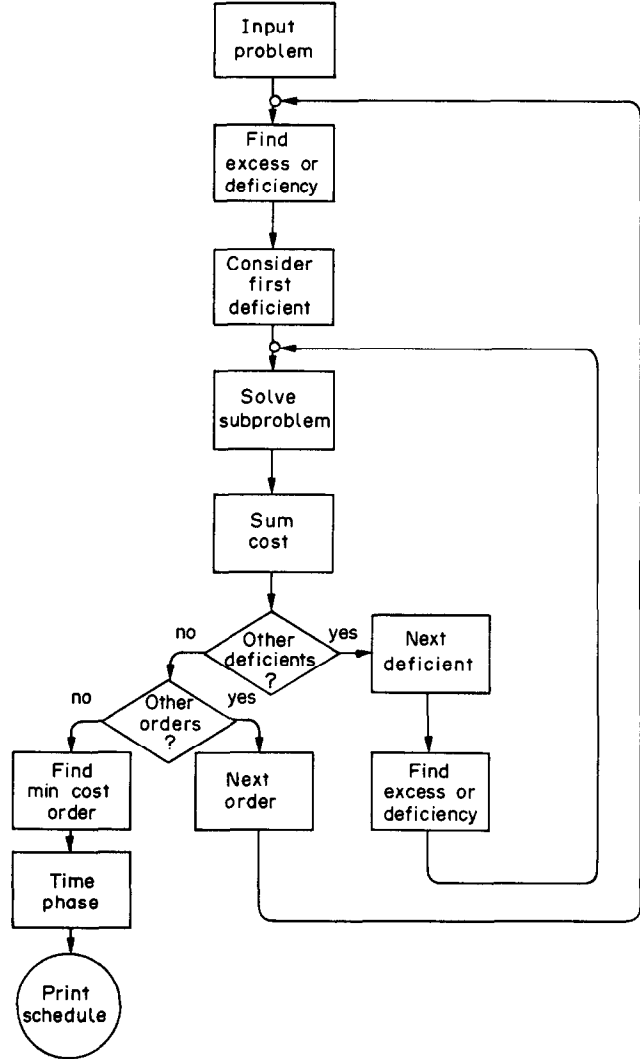


Fig. 2

and

$$Z_{irk} \in \{0, 1\}, \quad \delta_k \geq 0$$

where:

$$Z_{irk} = \begin{cases} 1 & \text{if the } r\text{th machine is switched from product } i \text{ to product } k, \\ 0 & \text{otherwise,} \end{cases}$$

- C_{ik} = the cost to switch a machine from product i to product k ,
- E_{irk} = capacity available on the r th machine of product i to be switched to product k ,
- Δ_k = the production deficiency of product k when the problem is formulated,
- δ_k = the surplus capacity of product k after solution of the current subproblem.

Note that the number of Z_{irk} variables is equal to the number of machines which can leave all products, i.e. $\sum_i L_i$.

Network interpretation of subproblems

These formulations can readily be viewed as min-cost-flow networks with gains. The set of source nodes corresponds to the set of machines currently setup for products with excess capacity. Each

product i creates L_i nodes. These source nodes have capacities of 1. For each problem formulated, the sink node corresponds to a product with a current deficiency. The sink capacity is the number of machine-days required to fill its production quota. For deficient product k , this is Δ_k . An arc which connects the r th machine capable of leaving the i th product with surplus has a gain, or multiplier E_{ir} , equal to the total amount of production which each it can supply to the deficient product. This amount can vary from one to H , the number of days in the horizon.

While helpful in visualizing the subproblem, the network described above is not strictly a min-cost-flow network with gains. The flow is not balanced. Unless there is only one deficient product, the supply will exceed the demand. This is a minor point and can be eliminated through the introduction of a dummy node. More significantly, the requirement that the flows be integer cannot be guaranteed in a generalized network. Therefore, no attempt has been made to exploit the special structure of this problem in finding its solution. The language associated with this network model will however, continue to be used to describe the problem.

Obtaining a solution

To implement the main step, an arbitrary order of the deficient products is chosen. The initial excess capacity is shipped to at least fill its demand, at the lowest setup cost. Note than in general, this demand will be overfilled, resulting in a new product with a surplus of capacity. The excess or deficiency associated with each product is now recalculated and a new ILP to supply the next product with a deficiency is solved. The total cost to supply all deficiencies is recorded. This set of steps is now repeated for each possible order of the deficient products and the minimum cost of all orders is chosen.

The chosen solution must finally, be converted to a time-phased set of setup changes. Consider the solution to the first subproblem. For each arc with flow (i.e. $Z_{irk} = 1$) determine the day, d_{ir} , to transfer the machine as, $d_{ir} = H - E_{ir} + 1$. Time-phasing of subsequent subproblems is identical.

An illustrative example

The following solves a randomly generated problem to illustrate the algorithm proposed above. The number of products (6), and length of the planning horizon (4), were chosen for convenience and the costs were equated to the distance between points, each corresponding to one of the products. The x - y coordinates of the points were generated from a uniform distribution. Thus costs satisfy the assumptions of symmetry and the triangle inequality. The cost to switch from any product i to a product j is displayed in Table 1. The number of machines initially setup on each product and the allocation of the production capacity to individual products were also randomly determined and are displayed in Table 2. Note that products 1, 2, 3 and 6 have excess capacity, while products 4 and 5 are deficient.

Table 1. Setup cost between products in dollars associated with example

	1	2	3	4	5	6
1	0	3.34	1.56	1.08	7.88	5.54
2	3.34	0	1.85	2.66	4.78	5.62
3	1.56	1.85	0	1.31	6.57	5.67
4	1.08	2.66	1.31	0	6.93	4.64
5	7.88	4.78	6.57	6.93	0	6.75
6	5.54	5.62	5.67	4.64	6.75	0

Table 2. Initial capacity and demand associated with example

Product number	Setup machines	Capacity	Demand (in machine-days)	Excess
1	5	20	9	11
2	5	20	18	2
3	5	20	15	5
4	3	12	21	-9
5	2	8	18	-10
6	6	24	23	1
Totals	26	104	104	0

A 0/1 ILP is now formulated to reallocate the excess capacity considering only one deficient product. The formulation is described through a network. First determine the number of machines that are candidates for setup changes during the planning horizon; each machine will be represented in the network as a source node. As seen in Table 2, product 1 has an excess capacity of 11 machine-days: enough to transfer three machines to other products. Therefore, three source nodes are created by product 1, and are illustrated in Fig. 3 as nodes 1A, 1B and 1C. Products 2, 3 and 6 are similarly analyzed. Next, one sink node, associated with a particular deficient product (arbitrarily, product 4 is used) is created with a capacity of Δ_4 the deficiency of product 4. Recall that this deficiency must be met or exceeded in a feasible solution.

Two numbers are associated with the arcs from the sources to the sink, the gain (or multiplier) and the cost. Costs are shown as arc numbers in parentheses and gains as arc numbers in square brackets on Fig. 3. The gain is the amount that each machine can produce during the horizon after subtracting any previously allocated capacity. For example, product 1 created three nodes which will supply a total of 11 units of capacity to other products. Two machines can be transferred at the start of the 4-day horizon, and therefore, have gains of four, while the third machine must produce product 1 for one day before transferring, and therefore has a gain of three. Costs associated with arcs to node 4 are the setup costs in Table 1. Finally, it should be emphasized, that flow on the arcs is restricted to the values zero and one.

Solution to this first subproblem satisfies the deficiency of product 4 by transferring three machines from product 1 at a cost of \$3.24. Figure 3 illustrates this by showing arcs with flow darker than other arcs. A total of eleven production units are transferred, so product 4 now is added to set of surplus products. Product 1 is now in balance.

The next subproblem is formulated to fill the deficiency of another product—in this example, only product 5 remains a candidate. Based on the solution to the first subproblem, a second one is formulated and presented graphically in Fig. 4. Its solution is obvious since only one feasible solution exists. The flow on each arc of Fig. 4 must equal one for a cost of \$31.60.

There are no more products with deficiencies, so a feasible solution to the main problem is at hand. Its cost is the sum of the subproblem costs or \$34.83. This cost is compared to the costs obtained when the deficient products are considered in the orders. For this example, only one other order is possible and its total cost is \$27.34, an improvement. The transfers associated with the two subproblems of the second order are summarized in Figs 5 and 6. The last step, time-phasing the subproblem solutions obtained from the best ordering of deficient products, is shown in Fig. 7. Figure 7 is a display similar to Fig. 1 except that only change-over arcs with positive flow have been included.

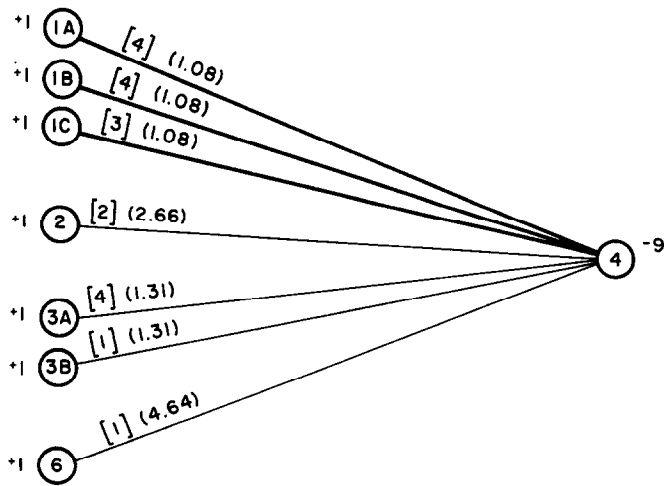


Fig. 3. Network of example subproblem 1 for first order.

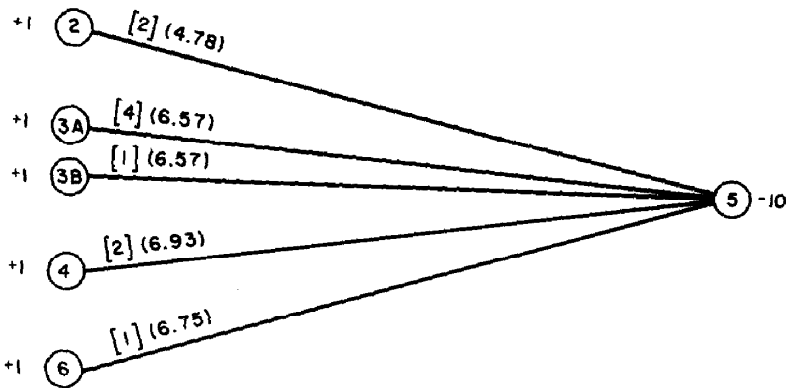


Fig. 4. Network of example subproblem 2 for first order.

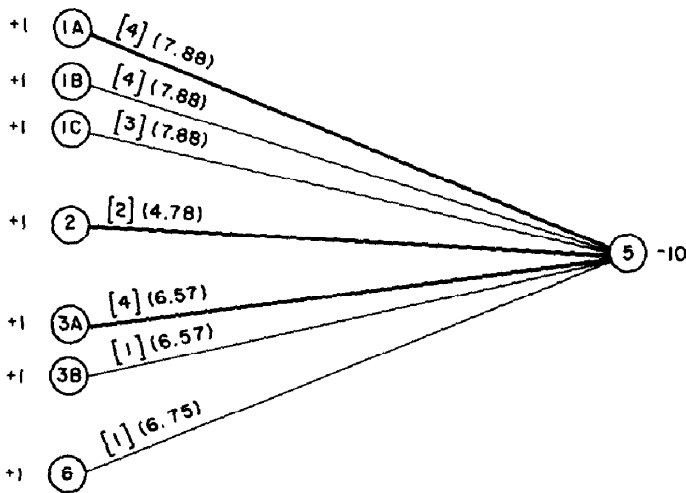


Fig. 5. Network of example subproblem 1 for second order.

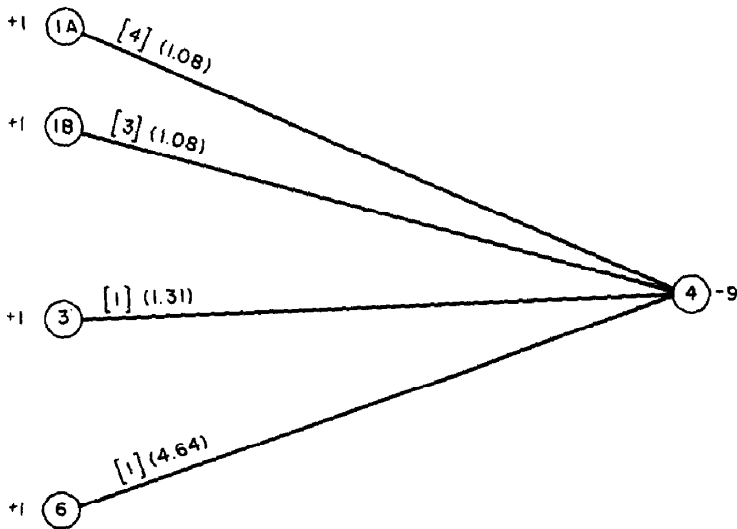


Fig. 6. Network of example subproblem 2 for second order.

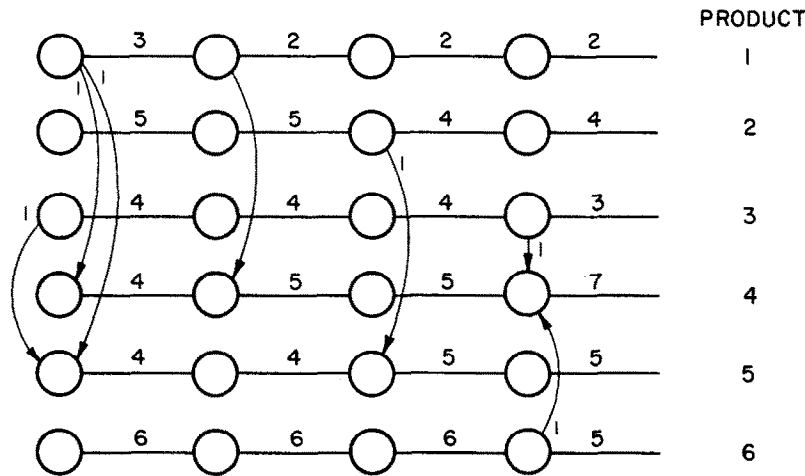


Fig. 7. Graph of optimal solution to example problem.

CONCLUSIONS

A new method for scheduling machines to minimize setup time has been presented. In many situations it represents an improvement over the model formulated by Dearing and Henderson [6]. The current algorithm reduces the problem by considering only resources that must be transferred. Therefore, the total number of products and the total number of machines to be scheduled do not limit use of the new method. This method is most appropriate for scheduling in firms that have a varying mix of established products and large sequence-dependent setup costs.

Computational limitations

Computational effort becomes prohibitive as the number of deficient products increases and as the deficiency size grows. Controlling the number of deficient products is essential because the number of subproblems increases exponentially with increasing deficient products. This may not be a problem in practice, judging from the small total number of products (or product families) reported by researchers studying situations of this type [1, 6, 7].

The second size limitation is more easily avoided. If the size of the deficiency is large, then a large number of machines must be transferred between products. Each potential machine transfer creates one subproblem 0/1 variable. While it is much easier to solve a 0/1 ILP than a general integer problem, it is nevertheless, essential to control this problem characteristic. Scaling may represent a feasible option if the number of variables is too large. For example, if several hundred machines are to be transferred to a few deficient products, it may be feasible to schedule blocks of ten machines, rather than individual machines. Some manual “fine tuning” of the solution may subsequently be necessary to exactly match product demands. Naturally, optimality will be destroyed.

Less restrictive assumptions

Four assumptions stated in the problem description may limit the use of this procedure. Two are easily relaxed while the remaining two suggest avenues for future research.

First consider the assumptions that production be measured in machine-days and that total production capacity match total demand. Changing production units merely changes the way the multipliers, E_{ij} , are calculated; a trivial change. To allow production capacity to exceed demand requires the creation of a dummy product to capture the excess capacity. The dummy product can be interpreted as switching a machine off without changing its setup and should have a zero setup cost.

It is more difficult to relax the assumption that no production is lost during a setup change. Yet, it is clear that some loss of production should occur in practice. Further, available personnel may limit the number of setup changes which can be performed during a single day. Modifying the algorithm to allow relaxation of these assumptions should be the subject of future research.

Solution quality

Finally, the algorithm has not been shown to produce an optimal solution (even when all assumptions hold). Empirical testing has been limited, due to the lack of a set of test problems with known optimal solutions. Every test problem which has been solved for an optimal solution using standard ILP techniques and the Dearing and Henderson formulation [6] has been resolved with identical results. But, these were necessarily very small problems. Future work should be completed to show the algorithm produces (or does not produce) optimal solutions.

REFERENCES

1. J. W. Barnes and J. J. Brennan, An improved algorithm for scheduling jobs on identical machines. *AIIE Trans.* **9**, No. 1 (1977).
2. S. E. Elmaghraby and S. H. Park, Scheduling jobs on a number of identical machines. *AIIE Trans.* **6**, No. 1 (1974).
3. M. H. Rothkopf, Scheduling independent tasks on parallel processors. *Mgmt Sci.* **12**, 437–446 (1966).
4. A. Dogramaci and J. Surkis, Evaluation of a heuristic for scheduling independent jobs on parallel identical processors. *Mgmt Sci.* **25**, 1208–1216 (1979).
5. M. D. Oliff, An integrated production planning model for multiproduct parallel processor environments. Ph.D. dissertation, Clemson University (1982).
6. P. M. Dearing and R. A. Henderson, Assigning looms in a textile weaving operation with changeover limitations. *Prod. Invent. Mgmt* **25**, 23–31 (1984).
7. E. E. Burch and R. T. Sumichrast, Using heirarchical techniques to schedule machines with simultaneous production capabilities. *Appl. Mgmt Sci.* In press.
8. E. S. Schwartz, A heuristic procedure for parallel sequencing with choice of machines. *Mgmt Sci.* **10**, 767–777 (1964).